Task 4 Numerical Methods

CARL-EMIL ROHDE
KVB125@ALUMNI.KU.DK

ACM Reference Format:
Carl-Emil Rohde, kvb125@alumni.ku.dk. 2018. Task 4 Numerical Methods. In *Woodstock '18: ACM Symposium on Neural Gaze Detection, June 03–05, 2018, Woodstock, NY*. ACM, New York, NY, USA, 5 pages. https://doi.org/XXXXXXX.XXXXXXX

PROBLEM 1

Traits	Bisection	Newton	Secant
Rate of Convergence	Linear	Quadratic	Superlinear
Per Iteration Cost	Low (Simple operations)	Moderate-High (Requires derivative)	Moderate (Requires two previous points)
Global Convergence	Yes, if initial brackets are chosen properly	No, only local	No, only local
Extends to Higher Dimensions	No (But there are multidimensional equivalents)	Yes (Newton's method for systems of equations)	No (But there are multidimensional equivalents)
Conditions for Convergence	Function must be continuous and different signs at endpoints	Function must be smooth and close to the root initially	Function must be smooth and two initial approximations are required

PROBLEM 2

3.1.2

a) According to the content on page 78 , the width of the interval at the n'th step is given by:

$$b_n - a_n = 2^{-n}\,(b_0 - a_0)$$

For the initial interval $[1.5, 3.5]$, $b_0 = 3.5$ and $a_0 = 1.5$. So, $b_0 - a_0 = 3.5 - 1.5 = 2$. Therefore, the width of the interval at the n'th step would be:

$$b_n - a_n = 2^{-n} \times 2 = 2^{-n+1}$$

b) The maximum distance between the root r and the midpoint c_n of the interval $[a_n, b_n]$ is given by:

$$|r - c_n| \leq \frac{1}{2}\,(b_n - a_n)$$

Using the expression for $b_n - a_n$ I derived before, this inequality becomes:

$$|r - c_n| \leq \frac{1}{2} \times 2^{-n+1} = 2^{-n}$$

Thus, the maximum distance possible between the root r and the midpoint of the interval at the n'th step is 2^{-n}.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Association for Computing Machinery.
Manuscript submitted to ACM

Carl-Emil Rohde
kvb125@alumni.ku.dk

3.1.9

For the case $[a_0, b_0] = [0, 1]$, the root r in binary representation can be expressed as $r = 0.d_1d_2d_3\ldots$ In decimal form, $r = \sum_{n=1}^{\infty} \frac{d_n}{2^n}$. For a general interval $[a_0, b_0]$, the root r is:

$$r = a_0 + (b_0 - a_0) \sum_{i=1}^{\infty} \frac{d_i}{2^i}$$

Here $d_n = 0$ if the interval at the n-th step is the left half of the previous interval, and $d_n = 1$ otherwise.

3.2.9

To solve for x_2 using Newton's method, I first need to find the derivative $f'(x)$ of $f(x) = x^3 - 2$. The derivative is $f'(x) = 3x^2$. Newton's method uses the formula:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

Starting with $x_0 = 1$, let's find x_1 and then x_2. To find x_1 :

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)}$$
$$x_1 = 1 - \frac{(1)^3 - 2}{3(1)^2}$$
$$x_1 = 1 - \frac{-1}{3}$$
$$x_1 = 1 + \frac{1}{3}$$
$$x_1 = \frac{4}{3}$$

To find x_2 :

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)}$$
$$x_2 = \frac{4}{3} - \frac{\frac{64}{27} - \frac{54}{27}}{3 \cdot \frac{16}{9}}$$
$$x_2 = \frac{4}{3} - \frac{10}{48}$$
$$x_2 = \frac{96}{72} - \frac{15}{72}$$
$$x_2 = \frac{9}{8}$$

$$x_2 = \frac{4}{3} - \frac{\left(\frac{4}{3}\right)^3 - 2}{3\left(\frac{4}{3}\right)^2}$$
$$x_2 = \frac{4}{3} - \frac{\frac{10}{27}}{\frac{48}{27}}$$
$$x_2 = \frac{4}{3} - \frac{5}{24}$$
$$x_2 = \frac{81}{72}$$

Thus, $x_2 = \frac{9}{8}$.

3.3.4

The formula for the secant method is given by:

$$x_{n+1} = x_n - f(x_n)\left[\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}\right]$$

To find x_2, I'll use $x_0 = 0$ and $x_1 = 1$ as x_{n-1} and x_n respectively in the formula. Firstly, I will evaluate $f(x_0)$ and $f(x_1)$:

$$f(x_0) = f(0) = 0^2 - 2 = -2$$

$$f(x_1) = f(1) = 1^2 - 2 = -1$$

Now, I substitute these into the formula to find x_2 :

$$x_2 = x_1 - f(x_1)\left[\frac{x_1 - x_0}{f(x_1) - f(x_0)}\right]$$

$$x_2 = 1 - (-1)\left[\frac{1-0}{-1-(-2)}\right]$$

$$x_2 = 1 + \frac{1}{1}$$

$$x_2 = 1 + 1$$

$$x_2 = 2$$

So, x_2 is 2 when we apply the secant method to the function $f(x) = x^2 - 2$ with $x_0 = 0$ and $x_1 = 1$.

3.5.1

I want to use Horner's Algorithm to evaluate $p(4)$, where the polynomial $p(z)$ is given by

$$p(z) = 3z^5 - 7z^4 - 5z^3 + z^2 - 8z + 2$$

We have the coefficients $a_0, a_1, \ldots, a_5$ of $p(z)$:

$$a_5 = 3, \quad a_4 = -7, \quad a_3 = -5, \quad a_2 = 1, \quad a_1 = -8, \quad a_0 = 2$$

I'll set $z_0 = 4$ and use Horner's algorithm to find $p(4)$. I arrange the calculation according to the tabular method suggested on page 112:

4	3	-7	-5	1	-8	2
		12	20	60	220	848
	3	5	15	61	212	850

Here's how the calculations were done step-by-step:

$$b_4 = a_5 = 3$$

$$b_3 = a_4 + z_0 b_4 = -7 + 4 \times 3 = 5$$

$$b_2 = a_3 + z_0 b_3 = -5 + 4 \times 5 = 15$$

$$b_1 = a_2 + z_0 b_2 = 1 + 4 \times 15 = 61$$

$$b_0 = a_1 + z_0 b_1 = -8 + 4 \times 61 = 212$$

$$p(4) = a_0 + z_0 b_0 = 2 + 4 \times 212 = 850$$

So, $p(4) = 850$

Additionally, the resulting polynomial $q(z)$ (of degree one less than $p(z)$) is:

$$q(z) = 3z^4 + 5z^3 + 15z^2 + 61z + 212$$

Carl-Emil Rohde
kvb125@alumni.ku.dk

Thus, can also represent $p(z)$ as:

$$p(z) = (z-4)q(z) + p(4) = (z-4)\left(3z^4 + 5z^3 + 15z^2 + 61z + 212\right) + 850$$

PROBLEM 3

The goal is to find an approximation to $\sqrt{7}$ with an accuracy of 10^{-7} using the secant method. I will model this as a root-finding problem.

The secant method is an iterative numerical method used for finding roots of a function $f(x)$. Given two initial guesses x_0 and x_1, the secant method iteratively refines these guesses using the formula:

$$x_{n+1} = x_n - \frac{f(x_n) \cdot (x_n - x_{n-1})}{f(x_n) - f(x_{n-1})}$$

The method stops when the absolute value of $f(x)$ is less than a given tolerance ϵ or the relative error between successive approximations is less than or equal to ϵ.

We start with initial guesses close to $\sqrt{7}$:

- $x_0 = 2.5$ (a bit less than $\sqrt{7}$)
- $x_1 = 3.0$ (a bit more than $\sqrt{7}$)

The tolerance level ϵ is set to 10^{-7} as per the problem statement.

The secant method was implemented to find the root of $f(x) = x^2 - 7$, using the initial guesses and tolerance level mentioned above.

After 4 iterations, the method converged to an approximate root of 2.64575131094764, meeting the specified accuracy level of 10^{-7}.

PROBLEM 4

To find the surface area S of a cone, the formula is:

$$S = \pi r^2 + \pi r l$$

Where r is the radius and l is the slant height. The slant height l can be related to the radius r and height h through the Pythagorean theorem:

$$l = \sqrt{r^2 + h^2}$$

Given that $S = 1200\ \text{m}^2$ and $h = 20$ m, I can plug these into the formulas:

$$1200 = \pi r^2 + \pi r\sqrt{r^2 + 400}$$

Now, I rewrite this equation as a root-finding problem $f(x) = 0$:

$$f(r) = \pi r^2 + \pi r\sqrt{r^2 + 400} - 1200 = 0$$

To solve the problem using the Secant method, I start with two initial guesses x_0 and x_1 for the radius r, and then iterate using the formula:

$$x_{n+1} = x_n - f(x_n)\,\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}$$

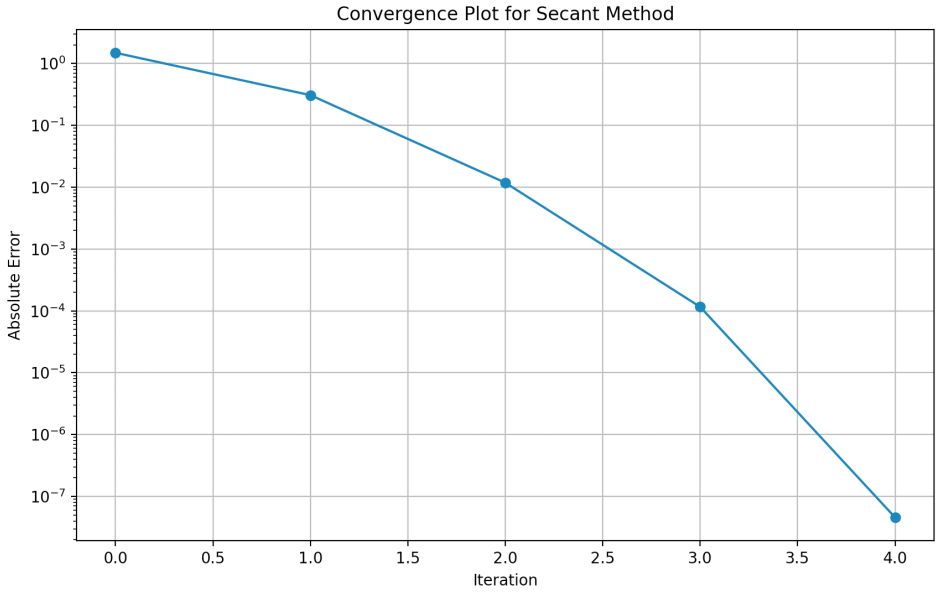
For the function $f(r) = \pi r^2 + \pi r\sqrt{r^2 + 400} - 1200$, I have implemented the Secant method in Python. When i run the code, i get that the approximate radius is 11,19, and the number of iterations to achieve the desired accuracy is 5.

Mathematically, super-linear convergence is often characterized as follows:

$$\lim_{n\to\infty} \frac{\epsilon_{n+1}}{\epsilon_n^p} = C$$

Here C is a constant and $p > 1$, but $p \neq 2$. To make a convergence plot, we want to track the absolute error $|\epsilon_n| = |x_n - x_{\text{root}}|$ at each iteration n.

I have implemented some Python code that does this, and produce the following plot:



We can see that the y-values (absolute error) decrease with each iteration. The rate of decrease is faster than a linear decrease (straight line on a log-linear plot) but slower than a quadratic decrease (concave down curve on a log-linear plot). Given these characteristics, we can confirm that the Secant method converges super-linearly for this problem.